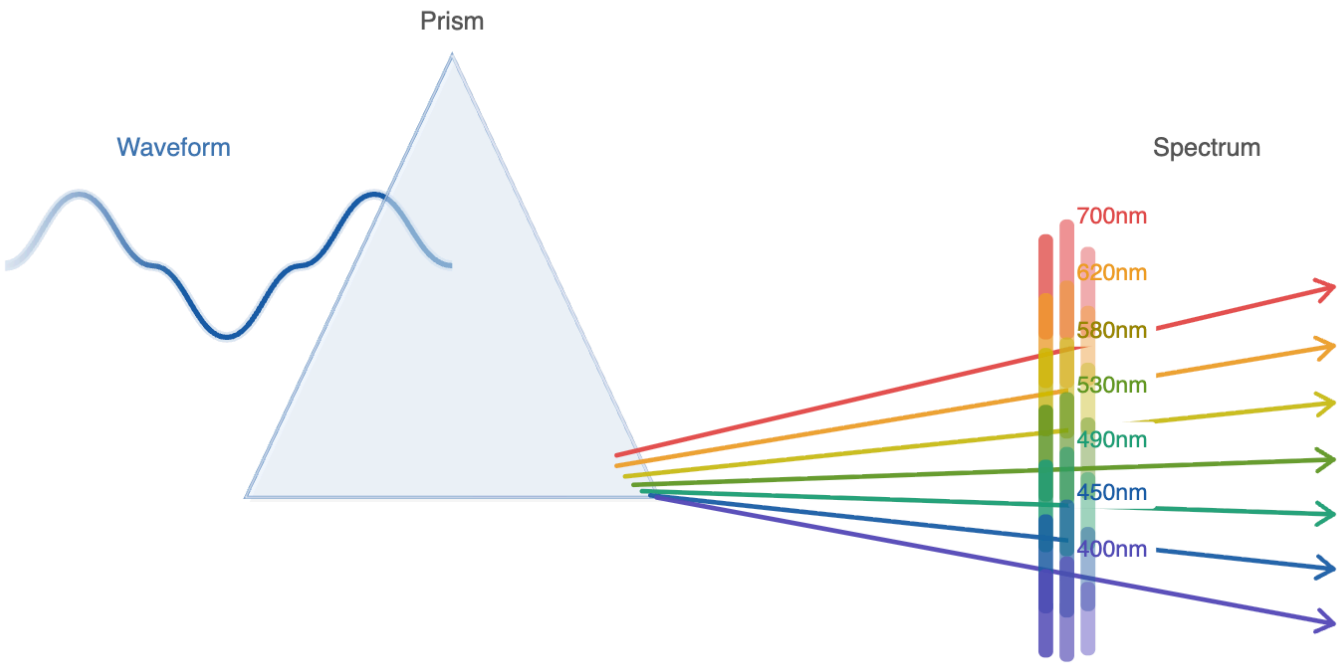
🪄 Digital Signal Processing

Lab 02: Audio Magic & The Time-Frequency Rift

"To understand the signal, you must look at it from another dimension."



🎯 Mission Objectives

สิ่งที่เราจะค้นหาในวันนี้:

1. **The Prism (FFT):** ถอดรหัสคลื่นเสียงที่ซับซ้อนให้กลายเป็นส่วนประกอบย่อยๆ
2. **The Glitch (Spectral Leakage):** ทำความเข้าใจ "รอยรั่ว" ของข้อมูลเมื่อเราตัดต่อสัญญาณ
3. **The Fix (Windowing):** วิธีอุดรอยรั่วเพื่อให้ได้ข้อมูลที่แม่นยำระดับวิศวกร
4. **The Map (Spectrogram):** การมองเห็นเสียงในรูปแบบภาพ 2 มิติ (เวลา vs ความถี่)
5. **The Spoof (Voice Disguise):** แฮ็กความเร็วเพื่อพรางเสียงตัวเอง

⏳ 1. Time vs. Frequency Domain

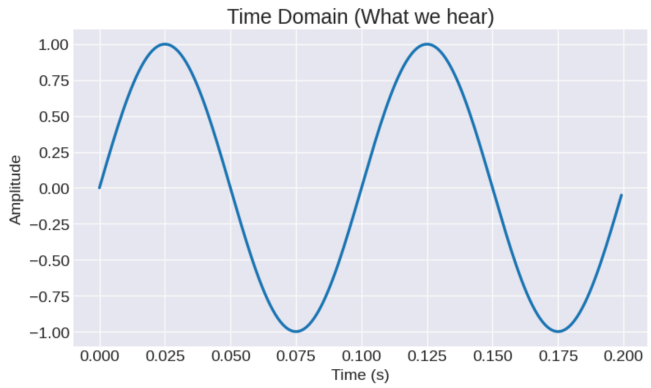
เรามองเห็นเสียงอย่างไร?

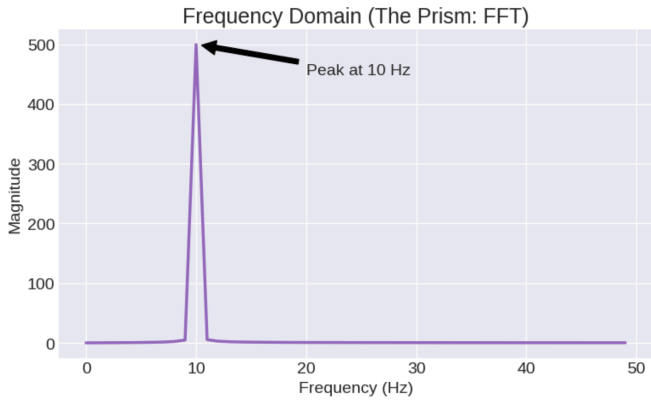
- **Time Domain (มิติของเวลา):** คือกราฟที่เราคุ้นเคย แกน X คือ "เวลา" แกน Y คือ "ความดัง"
  - *ข้อดี:* เห็นความสั้น-ยาว และจังหวะของเสียง
  - *ข้อเสีย:* เราไม่รู้เลยว่าในก้อนเสียงยุ่งๆ นั้น ประกอบด้วยโน้ต (ความถี่) อะไรบ้าง!

> *"ถ้า Time Domain คือสมูทตี้ปั่นรวมกัน Frequency Domain คือการแยกแยะว่ามีกล้วย สตรอว์เบอร์รี และส้ม อย่างละกี่เปอร์เซ็นต์"*

คำสั่ง โหลดไฟล์เสียง (.wav)

```
from scipy.io import wavfile
fs, data = wavfile.read('my_voice.wav') # ได้ Sampling Rate (fs) และข้อมูล Array
```

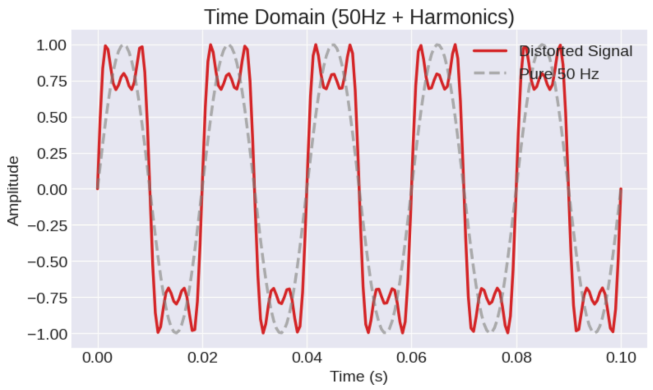


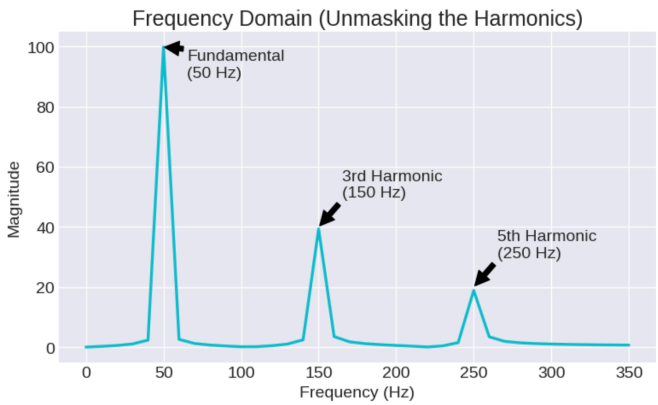


🌈 2. Enter the FFT (Fast Fourier Transform)

ปริซึมแยกแสงแห่งโลกดิจิทัล

- **FFT** คือสมการคณิตศาสตร์ที่ทำหน้าที่เหมือน "ปริซึม"
- เมื่อโยนคลื่นเสียงยุ่งๆ เข้าไป FFT จะแยกมันออกเป็น **Sine Wave บริสุทธิ์หลายๆ เส้น**
- **ผลลัพธ์ (Frequency Spectrum):** * แกน X = ความถี่ (Hz)
  - แกน Y = พลังงาน/ความดัง (Magnitude)
- *ยอดแหลม (Peak) ตรงไหน แปลว่าเสียงเรามีความถี่นั้นเยอะ!*





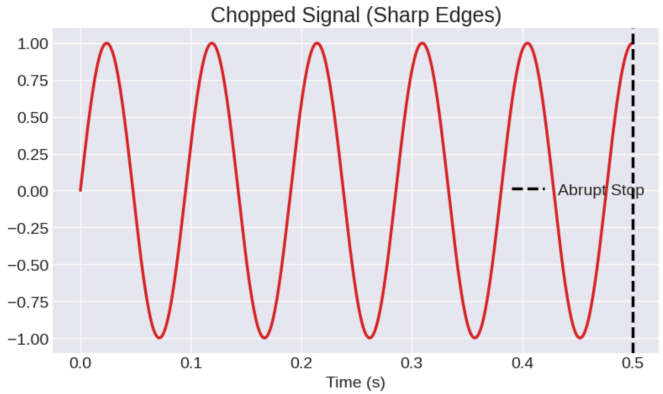
คำสั่ง FFT

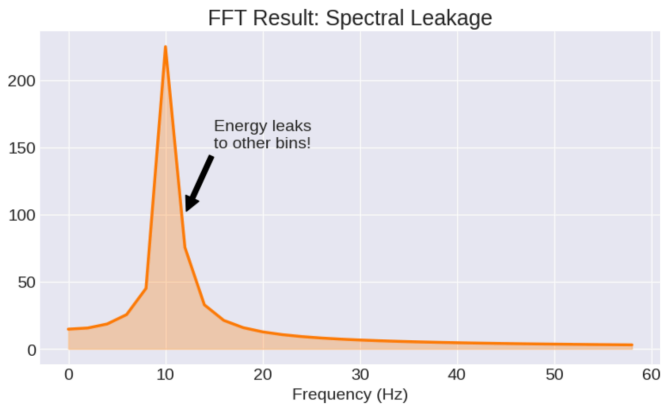
```
import numpy as np
# 1. สร้างแกนความถี่ X
freqs = np.fft.rfftfreq(len(data), 1/fs)
# 2. แปลงสัญญาณจาก Time -> Frequency
fft_values = np.abs(np.fft.rfft(data))
```

🚨 3. The Glitch: Spectral Leakage

เมื่อการ "ตัด" ทำให้ข้อมูลรั่วไหล

- คอมพิวเตอร์ไม่สามารถวิเคราะห์เสียงที่มีความยาวเป็นอนันต์ได้ เราจึงต้อง **"สับ (Chop)"** เสียงมาวิเคราะห์เป็นท่อนๆ
- **ปัญหา:** การสับเสียงแบบฉับพลัน ทำให้เกิดขอบที่คมกริบ (Sharp Edges) ที่จุดเริ่มต้นและจุดสิ้นสุดของท่อนนั้น
- **ผลเสีย (Leakage):** คณิตศาสตร์ของ FFT จะสับสนกับขอบที่คมกริบนี้ ทำให้พลังงานของความถี่หลัก **"รั่ว"** ออกไปด้านข้าง
- **อาการที่พบ:** แทนที่จะได้ยอดกราฟ (Peak) แหลมๆ แม่นยำ เรากลับได้ก้อนกราฟเบลอๆ บานๆ (Smudged Peaks)

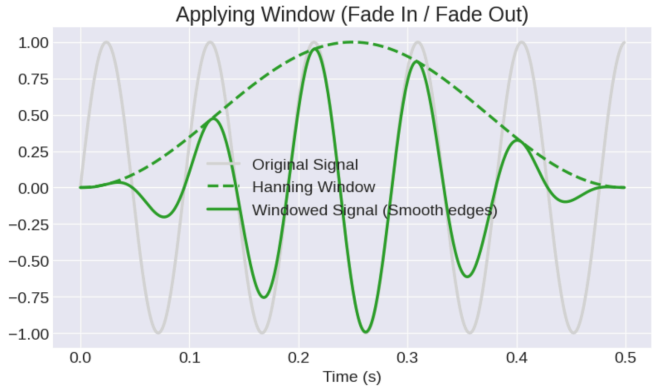


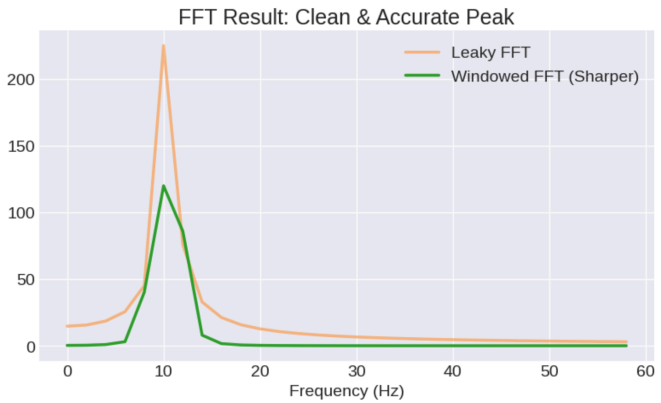


🪟 4. The Fix: Windowing

ลบเหลี่ยมคม ด้วยการใส่หน้าต่าง

- **วิธีแก้:** ก่อนทำ FFT เราจะเอา **Window Function** (เช่น หน้าต่างแบบ Hanning) ไปคูณกับก้อนเสียงนั้น
- **มันทำงานอย่างไร?:** หน้าต่างนี้จะค่อยๆ **Fade in (เสียงค่อยๆ ดังขึ้น)** ที่ตอนต้น และ **Fade out (เสียงค่อยๆ เบาลง)** ที่ตอนท้าย
- **ผลลัพธ์:** ขอบที่คมกริบจะถูกบีบให้สมูทกลายเป็น 0
- **ประโยชน์:** พลังงานไม่รั่วไหล ยอดกราฟ FFT กลับมาแหลมเปี๊ยบและแม่นยำระดับวิศวกร!





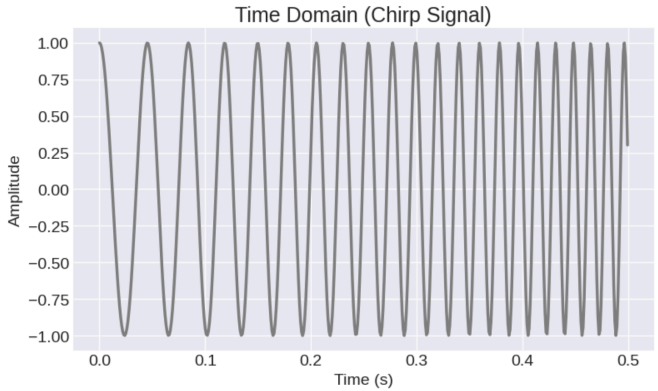
🛠️ The Fix: Windowing (การใส่หน้าต่าง)

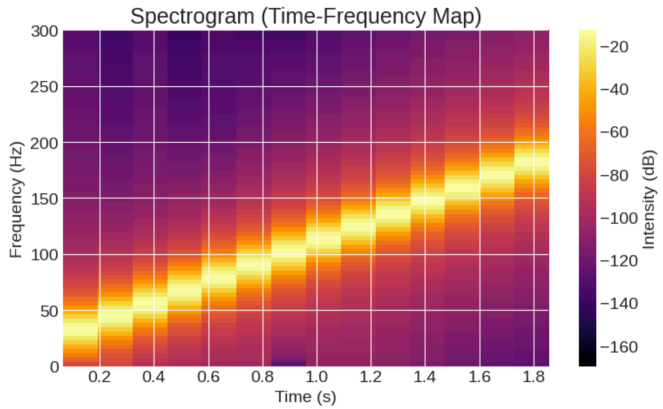
```
window = np.hanning(len(data))    # สร้างหน้าต่างทรงระฆังคว่ำ
windowed_data = data * window     # นำไปคูณเพื่อลบเหลี่ยมมุม (Fade in/out)
```

🗺️ 5. The Map: Spectrogram

เมื่อกราฟ FFT บอกไม่ได้ว่า "เกิดขึ้นเมื่อไหร่"

- **จุดอ่อนของ FFT:** มันบอกได้ว่ามีความถี่อะไรบ้างในไฟล์เสียง *แต่บอกไม่ได้ว่าความถี่นั้นเกิดขึ้นในวินาทีที่เท่าไหร่*
- **Spectrogram คือคำตอบ!** มันคือการเอา FFT เล็กๆ หลายๆ อันมาเรียงต่อกันตามเวลา
- **การอ่าน Spectrogram (ภาพ 2 มิติ):**
  - **แกน X:** เวลา (Time)
  - **แกน Y:** ความถี่ (Frequency)
  - **สี (Color):** ความสว่าง/ความเข้ม บ่งบอกถึงความดัง (Amplitude)





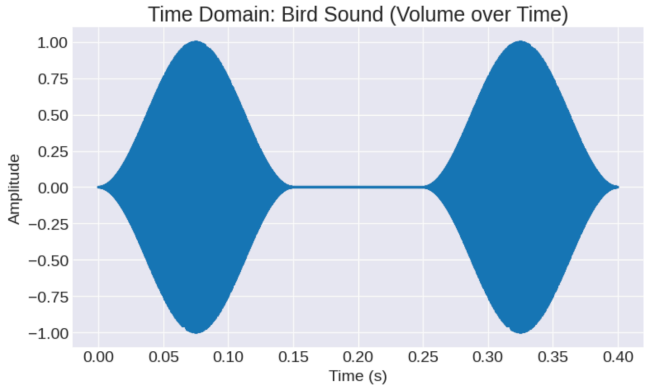
💻 The Syntax:

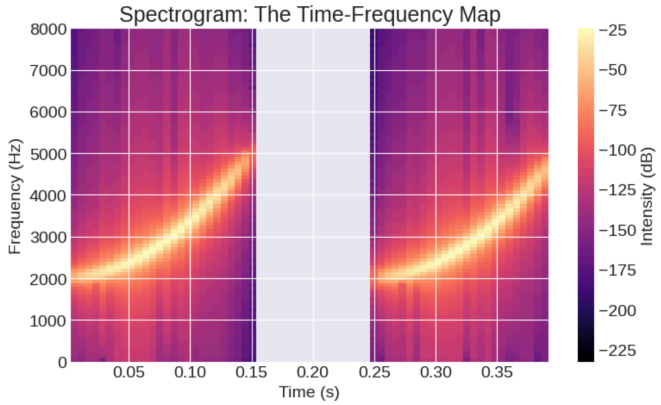
```
import matplotlib.pyplot as plt
# NFFT คือขนาดหน้าต่างย่อย (ยิ่งน้อยยิ่งเห็นเวลาชัด แต่ความถี่เบลอ)
plt.specgram(x=data, NFFT=1024, Fs=fs, cmap='inferno')
```

🐦 6. The Ultimate Map: Bird Chirp

มองเห็นจังหวะและทำนองผ่าน Spectrogram

- ลองนึกถึงเสียงนกร้อง **"จิ๊บ... จิ๊บ..."** ซึ่งเป็นเสียงที่มีความถี่กวาดจากต่ำไปสูงอย่างรวดเร็ว
- **Time Domain (กราฟบน):** เราจะเห็นเพียงกลุ่มก้อนพลังงานเสียงที่เกิดขึ้นแล้วดับไป เป็นจังหวะ แต่ไม่สามารถรับรู้ถึง "ความแหลม" หรือทำนองของเสียงได้
- **Spectrogram (กราฟล่าง):** เผยให้เห็นรูปแบบที่ซ่อนอยู่! เราจะเห็นลักษณะความถี่ที่พุ่งทะยานขึ้น (ตวัดขึ้น) ในแต่ละจังหวะการร้องของนกได้อย่างชัดเจน
- *นี่คือประโยชน์สูงสุดของ Spectrogram ที่เชื่อมโยงโสตประสาทการได้ยิน เข้ากับการมองเห็นได้อย่างสมบูรณ์แบบ*





🚀 Mission Briefing (Lab 02)

ลำดับการแฮ็กข้อมูลในวันนี้

1. อัดเสียงตัวเองพูดว่า *"Digital Signal Processing"* แล้วอัปโหลดเข้า Colab (`.wav`)
2. โหลดเสียงด้วย `scipy.io.wavfile` และจัดการให้เป็น Mono
3. ทำ **FFT** เพื่อดูกราฟความถี่ของเสียงตัวเอง
4. ลองสร้าง **Hanning Window** มาครอบเสียง แล้วเปรียบเทียบกราฟเพื่อดูการแก้ปัญหา Spectral Leakage
5. สร้าง **Spectrogram** เพื่อดูความลับของเสียงแบบ 2 มิติ
6. พรางเสียงตัวเองเป็น Darth Vader ด้วยการแฮ็ก Sampling Rate (`fs`)

Ready? Open your Colab and Let's Analyze! 👩‍💻👨‍💻